

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
import os
print(os.getcwd())
```

 C:\Users\hp\Desktop\PROJECTS\election_analysis

```
df=pd.read_csv("election_results_2024-checkpoint.csv")
```

```
df.info()
```

 <class 'pandas.core.frame.DataFrame'>
RangeIndex: 543 entries, 0 to 542
Data columns (total 8 columns):
Column Non-Null Count Dtype

0 Constituency 543 non-null object
1 Const. No. 543 non-null int64
2 Leading Candidate 543 non-null object
3 Leading Party 543 non-null object
4 Trailing Candidate 542 non-null object
5 Trailing Party 542 non-null object
6 Margin 543 non-null object
7 Status 543 non-null object
dtypes: int64(1), object(7)
memory usage: 34.1+ KB

```
df.head()
```

	Constituency	Const. No.	Leading Candidate	Leading Party	Trailing Candidate	Trailing Party	Margin	Status
0	AJMER	13	BHAGIRATH CHOUDHARY	Bharatiya Janata Party	RAMCHANDRA CHOUDHARY	Indian National Congress	329991	Result Declared
1	ALWAR	8	BHUPENDER YADAV	Bharatiya Janata Party	LALIT YADAV	Indian National Congress	48282	Result Declared
2	AMBALA	1	VARUN CHAUDHRY	Indian National Congress	BANTO KATARIA	Bharatiya Janata Party	49036	Result Declared

```
df.dtypes, df.index.dtype
```

 (Constituency object
Const. No. int64
Leading Candidate object
Leading Party object
Trailing Candidate object
Trailing Party object
Margin object
Status object
dtype: object,
dtype('int64'))

```
party_votes = df.groupby('Leading Party')['Margin'].sum()
df['Margin'] = pd.to_numeric(df['Margin'], errors='coerce')
```

```
# Party with highest and lowest margin of victory
highest_margin = df.loc[df['Margin'].idxmax()]
lowest_margin = df.loc[df['Margin'].idxmin()]
```

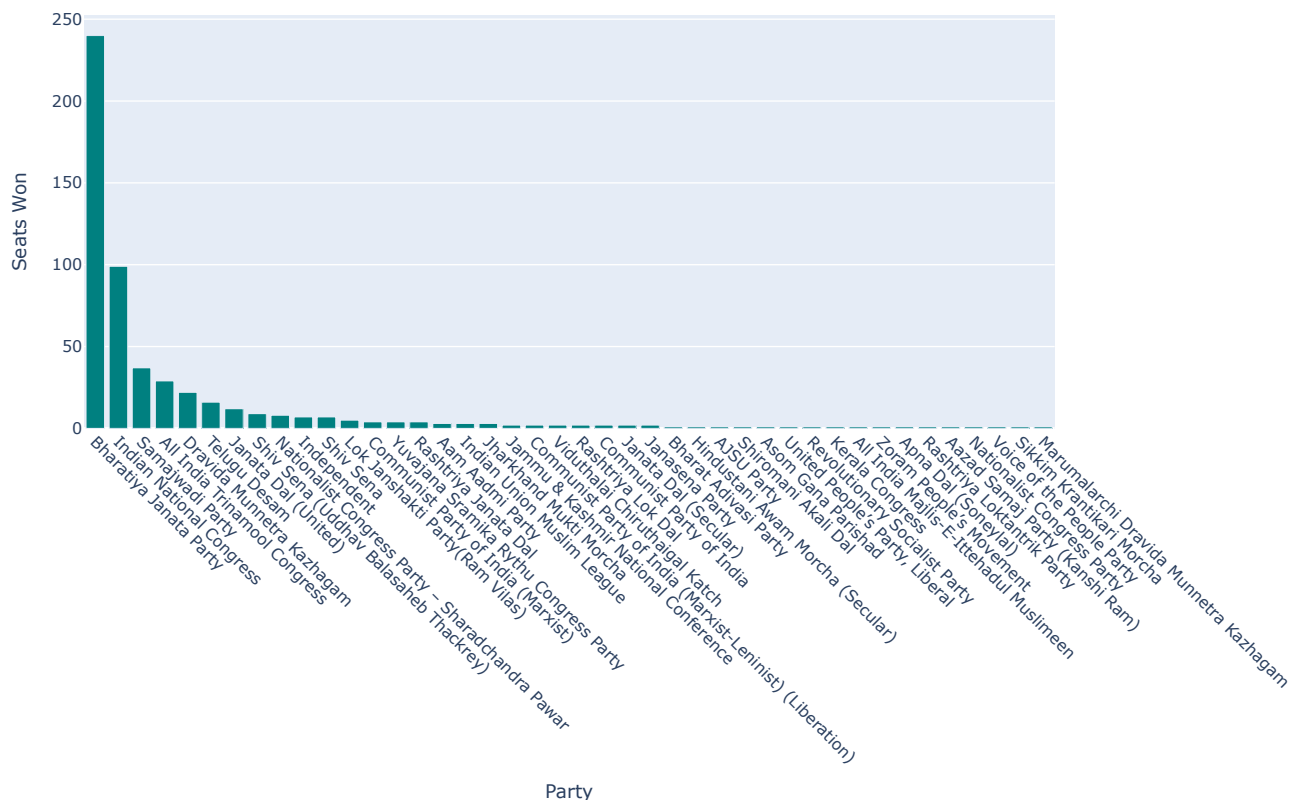
1: parties and seats they won

```
# highest and lowest votes
leading_party_highest_votes = party_votes.idxmax()
leading_party_lowest_votes = party_votes.idxmin()

# Number of seats won by each party
seats_won = df['Leading Party'].value_counts().sort_values(ascending=False)

fig = go.Figure(
    data=[go.Bar(x=seats_won.index, y=seats_won.values, marker_color='teal')],
    layout_title_text="Number of Seats Won by Each Party"
)
```

```
fig.show()
```



```

rahul_entries = df[df['Leading Candidate'] == 'RAHUL GANDHI']
modi_entries = df[df['Leading Candidate'] == 'NARENDRA MODI']
amit_entries = df[df['Leading Candidate'] == 'AMIT SHAH']

# Get the votes for Rahul Gandhi, Narendra Modi, and Amit Shah
rahul_votes = rahul_entries['Margin'].values
modi_votes = modi_entries['Margin'].values[0] if not modi_entries.empty else 0
amit_votes = amit_entries['Margin'].values[0] if not amit_entries.empty else 0

# Get the original constituency names for Rahul Gandhi
rahul_constituencies = list(rahul_entries['Constituency'])

# Get the original constituency name for Narendra Modi
modi_constituency = modi_entries['Constituency'].values[0] if not modi_entries.empty else "Modi Constituency"

# Get the original constituency name for Amit Shah
amit_constituency = amit_entries['Constituency'].values[0] if not amit_entries.empty else "Amit Shah Constituency"

# Combine the data
data_to_plot = pd.DataFrame({
    'Candidate': ['Rahul Gandhi'] * len(rahul_votes) + ['Narendra Modi', 'Amit Shah'],
    'Constituency': rahul_constituencies + [modi_constituency, amit_constituency],
    'Votes': list(rahul_votes) + [modi_votes, amit_votes]
})

# Plot the comparison

```

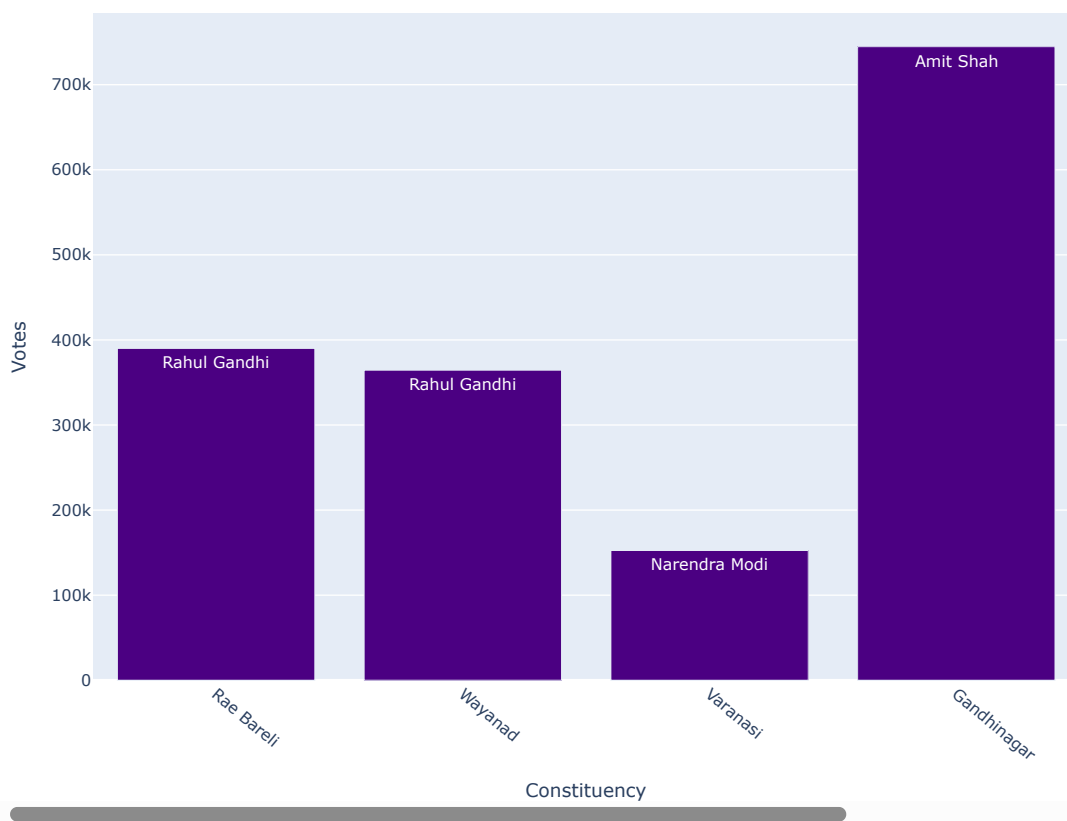
```
fig = go.Figure(
    data=[
        go.Bar(
            x=data_to_plot['Constituency'],
            y=data_to_plot['Votes'],
            text=data_to_plot['Candidate'],
            marker_color='indigo'
        )
    ],
    layout_title_text="Votes Received by Top Candidates"
)

fig.update_layout(
    xaxis_title="Constituency",
    yaxis_title="Votes",
    title_font_size=20,
    xaxis_tickangle=40,
    height=700,
    width=900
)

fig.show()
```



Votes Received by Top Candidates



3: Highest and lowest Victory Candidate

```
highest_margin_entry = df.loc[df['Margin'].idxmax()]
lowest_margin_entry = df.loc[df['Margin'].idxmin()]

# Combine the data
data_to_plot = pd.DataFrame({
    'Candidate': [highest_margin_entry['Leading Candidate'], lowest_margin_entry['Leading Candidate']],
    'Party': [highest_margin_entry['Leading Party'], lowest_margin_entry['Leading Party']],
    'Margin': [highest_margin_entry['Margin'], lowest_margin_entry['Margin']]
})

# Plot the comparison
fig = go.Figure(
    data=[
        go.Bar(
            x=data_to_plot['Party'],
            y=data_to_plot['Margin'],
            text=data_to_plot['Candidate'],

```

```

        marker_color='indigo'
    )
],
layout_title_text="Votes Received by Top Candidates"
)

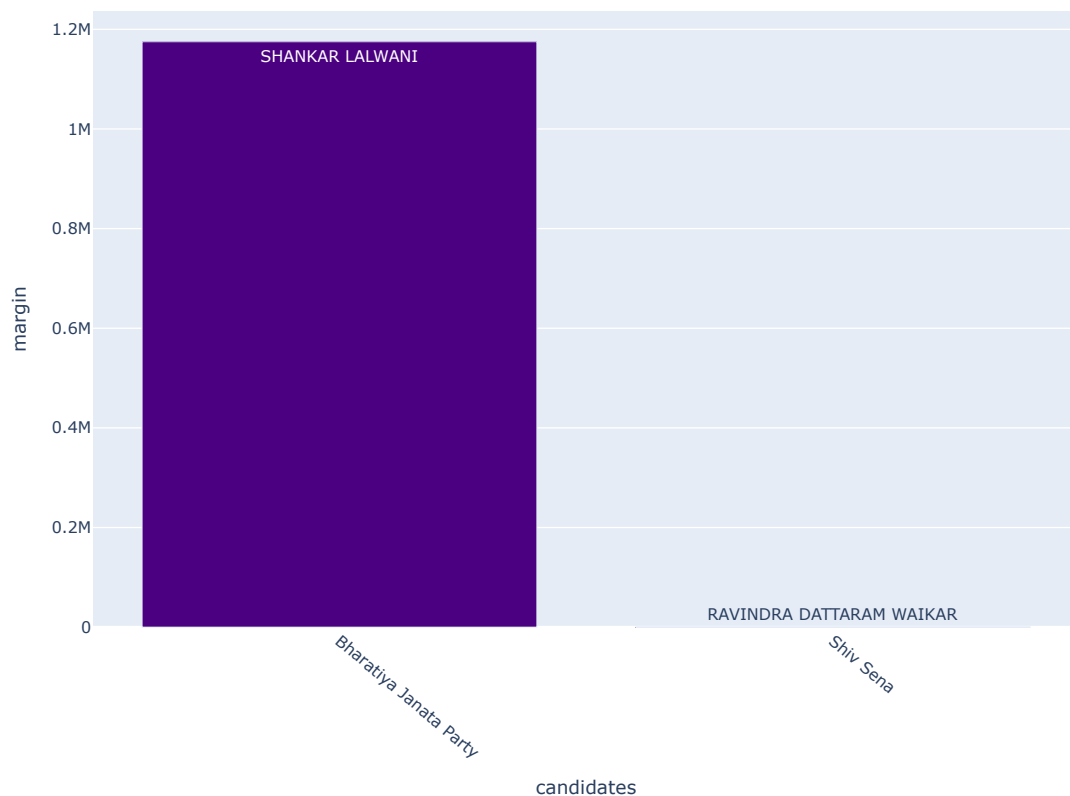
fig.update_layout(
    xaxis_title="candidates",
    yaxis_title="margin",
    title_font_size=20,
    xaxis_tickangle=40,
    height=700,
    width=900
)

fig.show()

```



Votes Received by Top Candidates



```

fig = px.histogram(
    df,
    x='Margin',
    nbins=20,
    title='Histogram of Margin of Victory',
    labels={'Margin': 'Margin of Victory'},
    marginal='rug',
    opacity=0.75
)

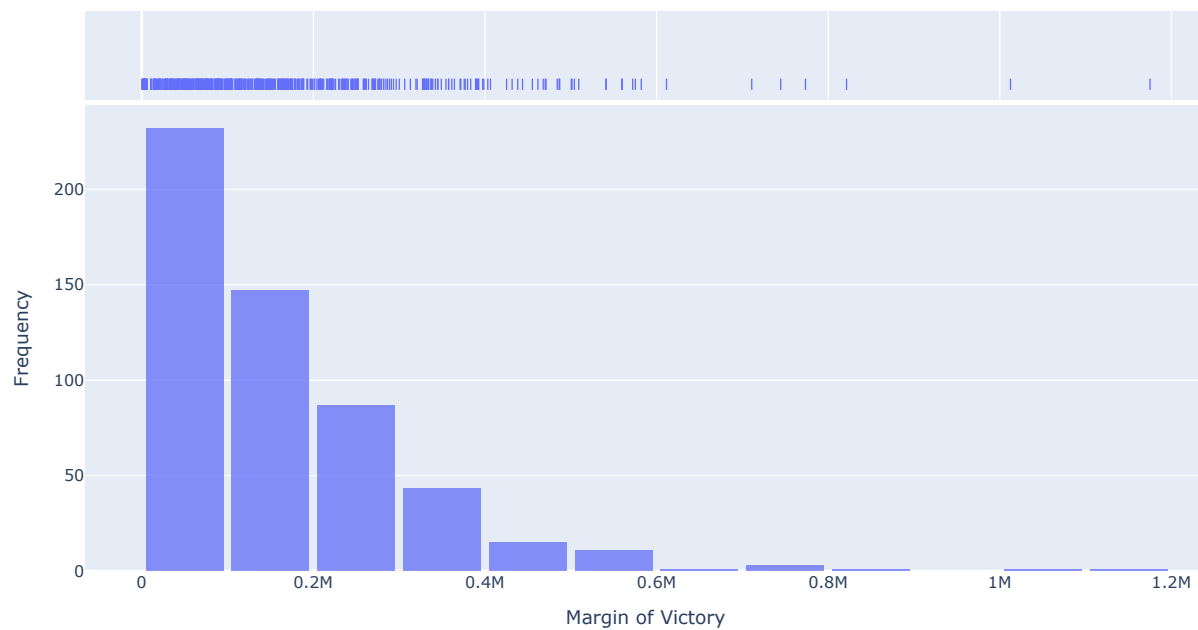
fig.update_layout(
    xaxis_title='Margin of Victory',
    yaxis_title='Frequency',
    bargap=0.1,
    height=600,
    width=1000
)

fig.show()

```



Histogram of Margin of Victory



4: Votes distribution by party

```

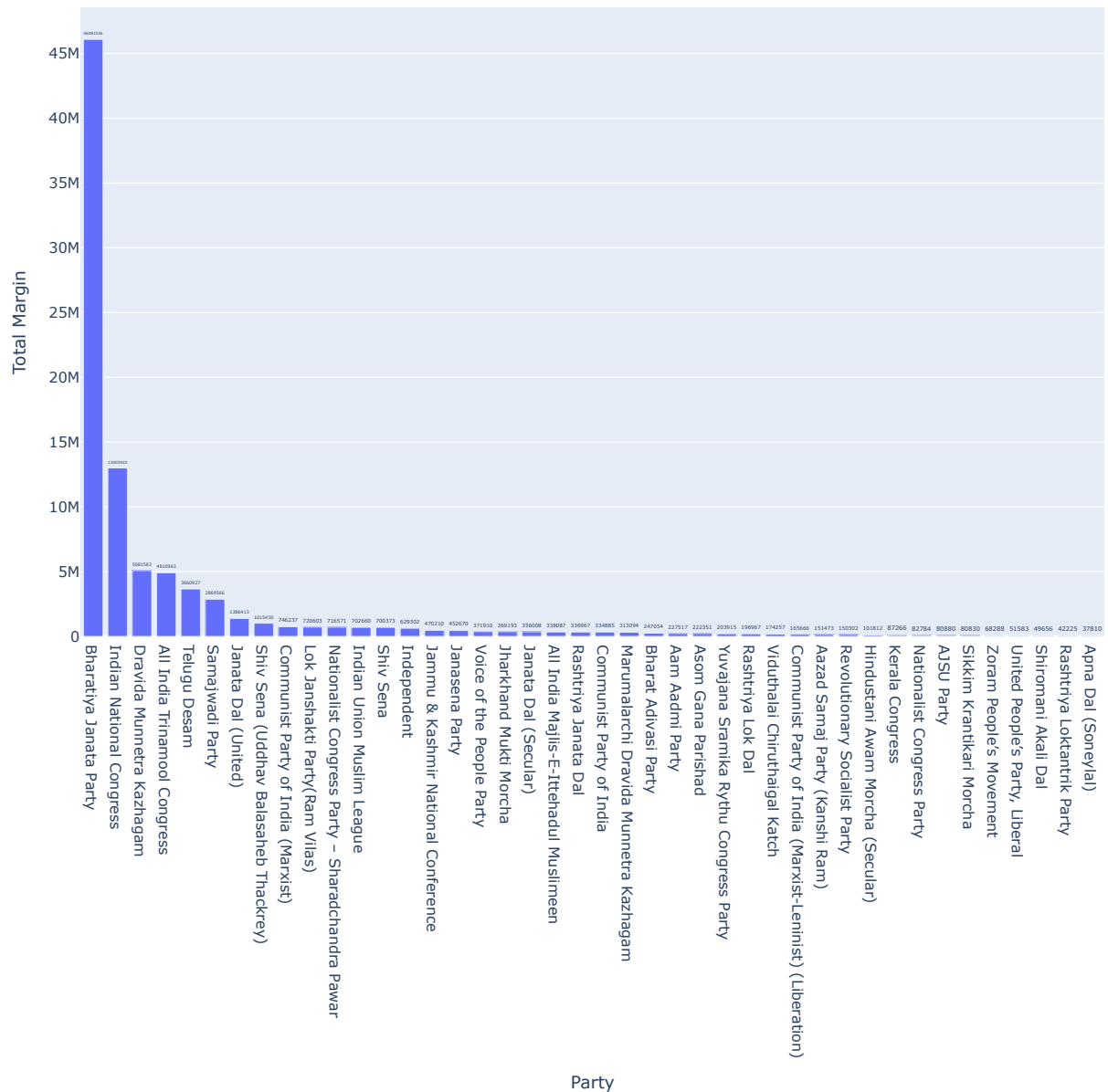
party_votes = df.groupby('Leading Party')['Margin'].sum().sort_values(ascending=False)
#
# Plot chart
fig = px.bar(
    x=party_votes.index,
    y=party_votes.values,
    labels={'x': 'Party', 'y': 'Total Margin'},
    title='Total Margin of Victory by Leading Party',
    text=party_votes.values
)

fig.update_traces(textposition='outside')
fig.update_layout(height=1000, width=1000)
fig.show()

```



Total Margin of Victory by Leading Party



5: Top 10 trailing party by VOTE

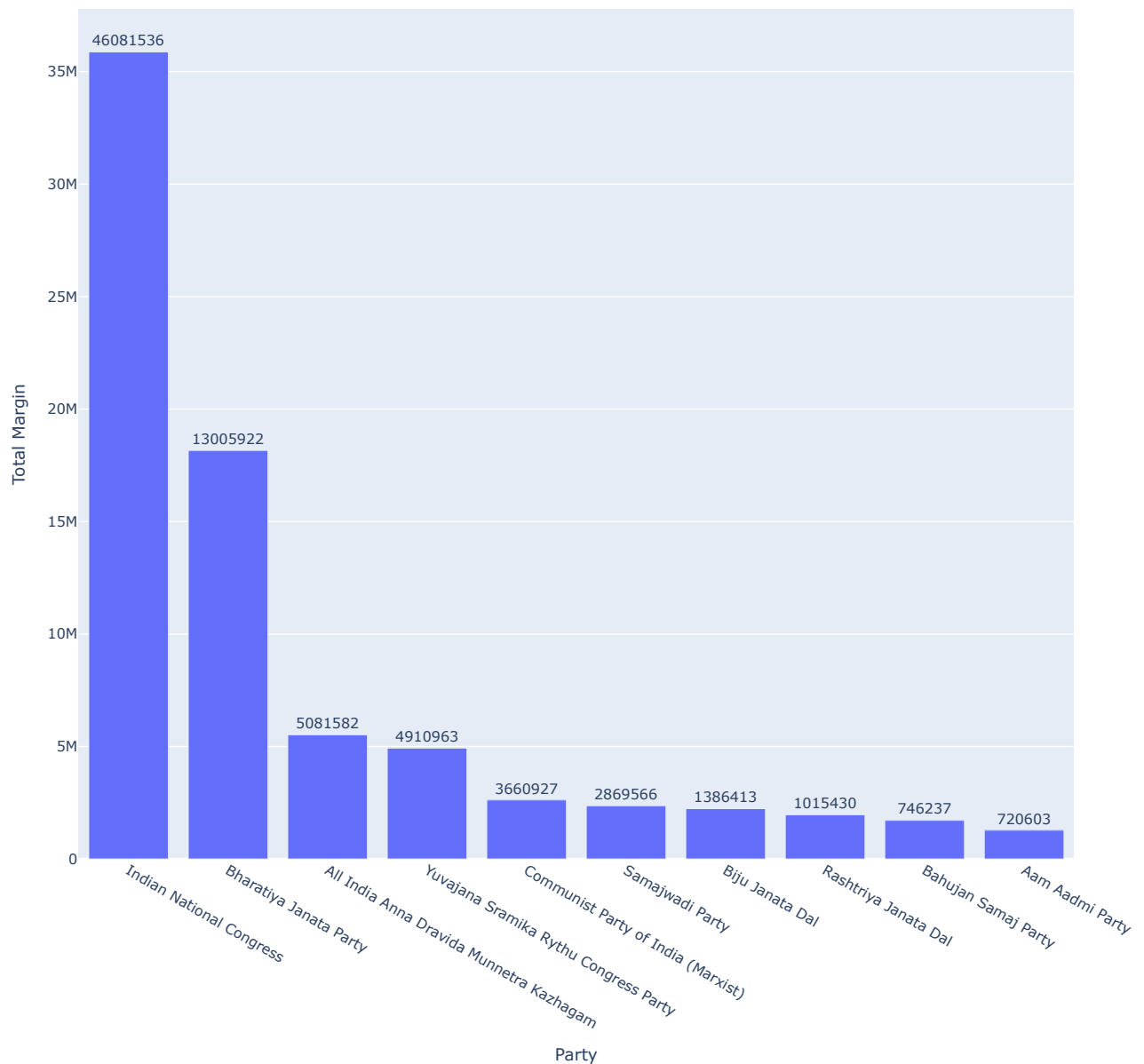
```
trailing_party_votes = df.groupby('Trailing Party')['Margin'].sum().sort_values(ascending=False)
trailing_party_seats = df['Trailing Party'].value_counts()
```

```
fig = px.bar(
    x=trailing_party_votes.index[:10],
    y=trailing_party_votes.values[:10],
    labels={'x': 'Party', 'y': 'Total Margin'},
    title='Trailing parties as per votes',
    text=party_votes.values[:10]
)
```

```
fig.update_traces(textposition='outside')
fig.update_layout(height=1000, width=1000)
fig.show()
```



Trailing parties as per votes



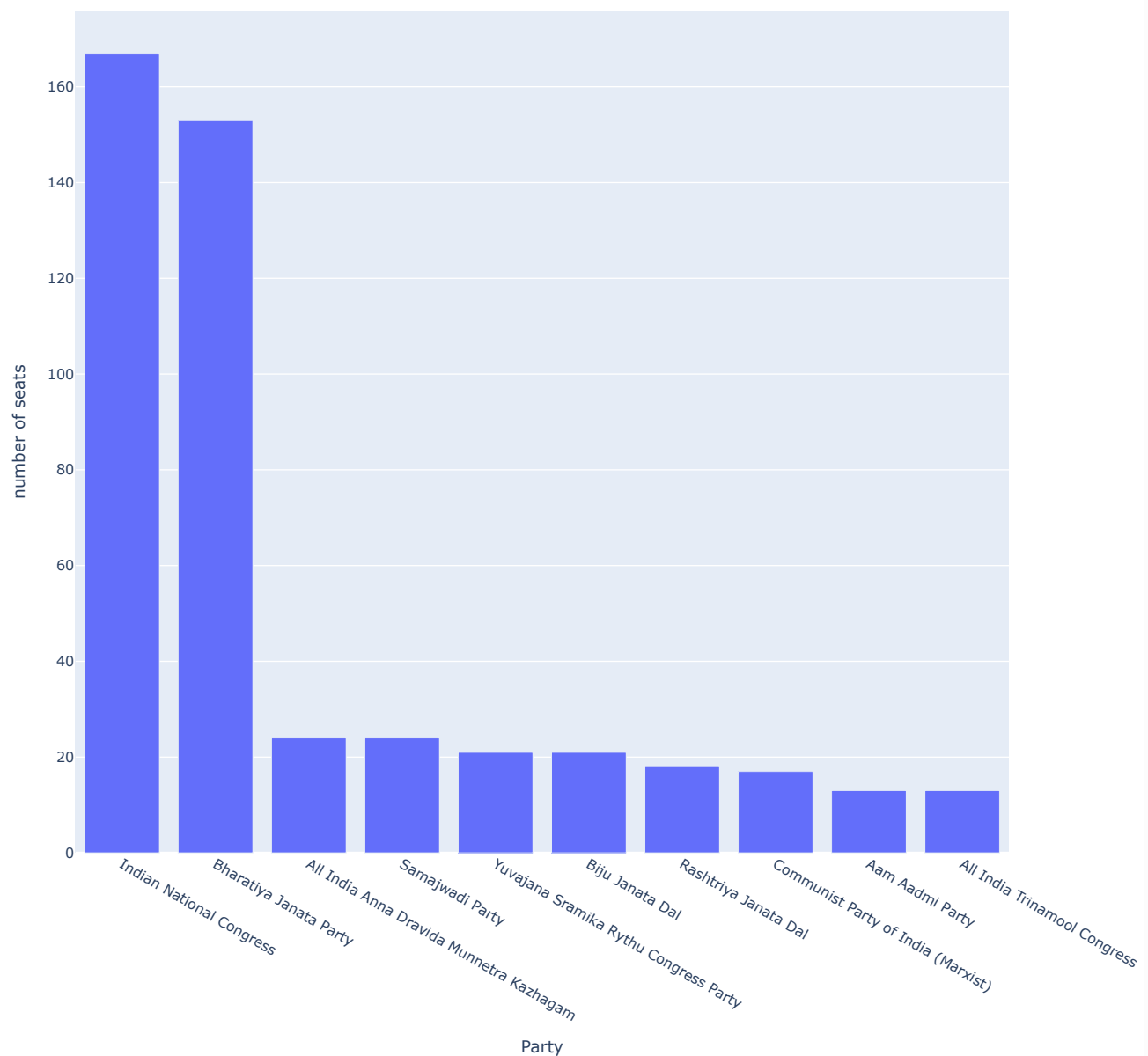
6: Top 10 trailing party by SEAT

```
fig = px.bar(
    x=trailing_party_seats.index[:10],
    y=trailing_party_seats.values[:10],
    labels={'x': 'Party', 'y': 'number of seats'},
    title='Trailing parties as per seats',
)

fig.update_traces(textposition='outside')
fig.update_layout(height=1000, width=1000)
fig.show()
```



Trailing parties as per seats



7: Top 10 Leading party by Votes

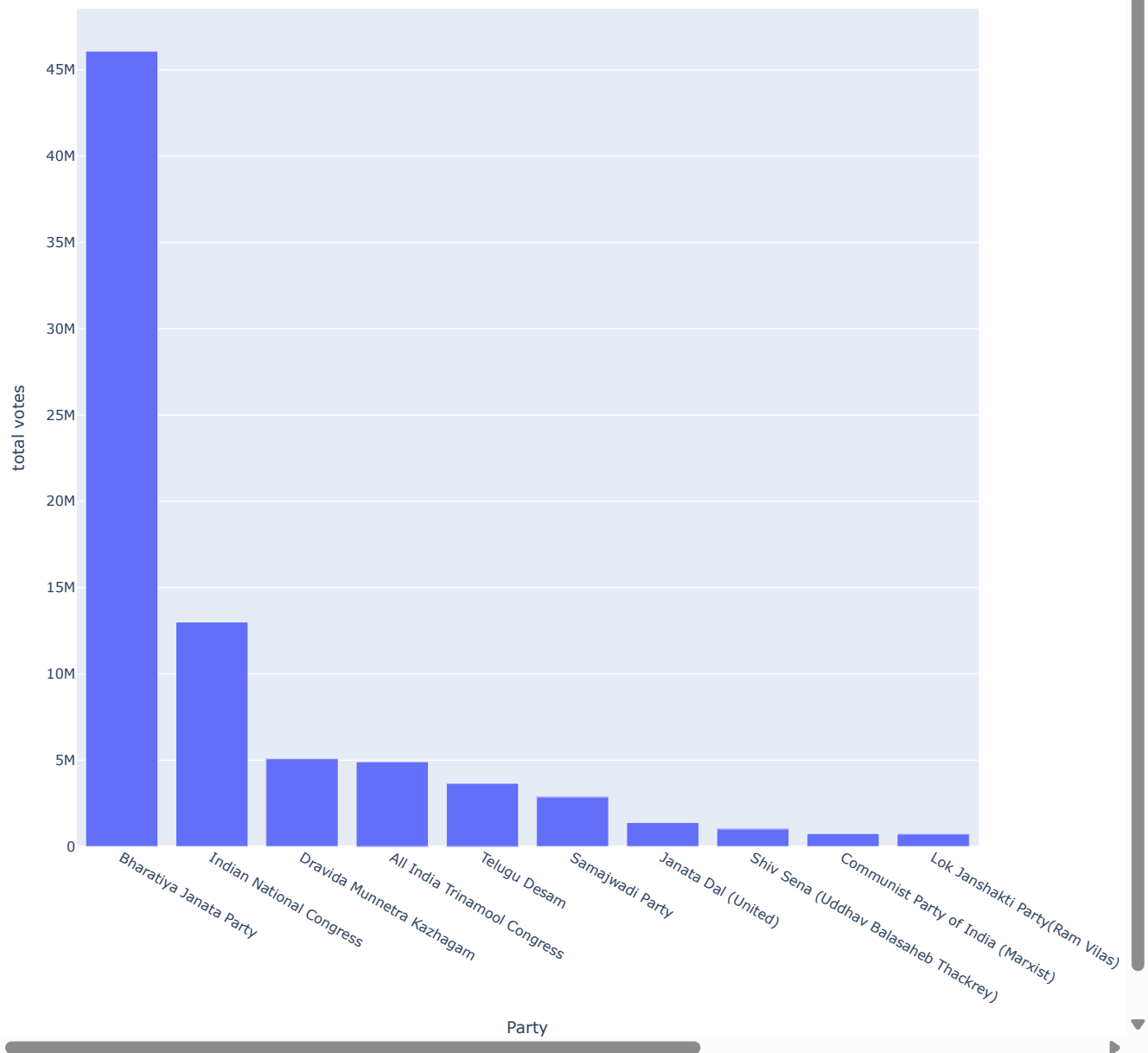
```
leading_party_votes = df.groupby('Leading Party')['Margin'].sum().sort_values(ascending=False)
leading_party_seats = df['Leading Party'].value_counts()
```

```
fig = px.bar(
    x=leading_party_votes.index[:10],
    y=leading_party_votes.values[:10],
    labels={'x': 'Party', 'y': 'total votes'},
    title='leading parties as per votes',
)

fig.update_traces(textposition='outside')
fig.update_layout(height=1000, width=1000)
fig.show()
```




leading parties as per votes



8: Top 10 leading party by Seats

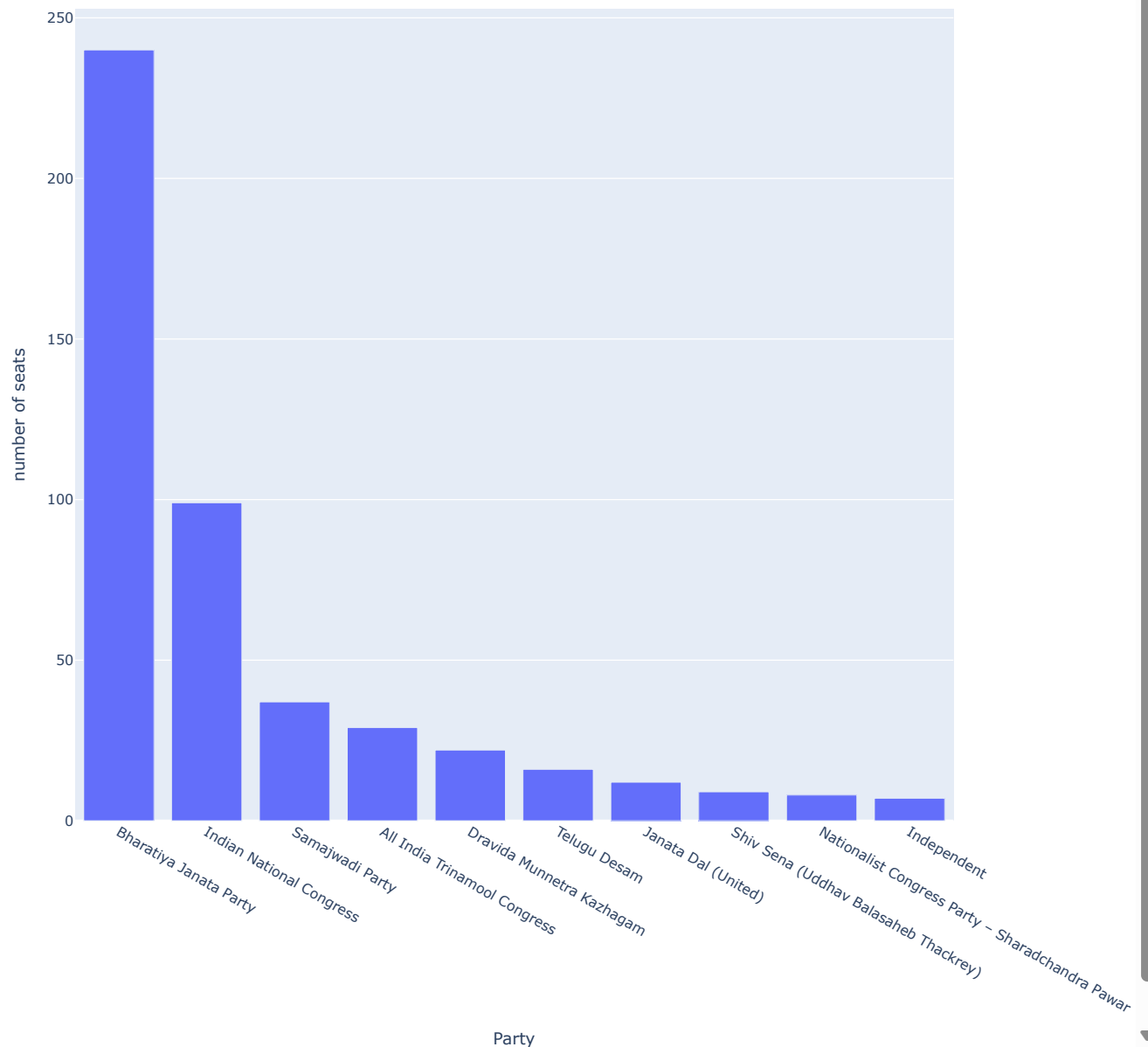
```
leading_party_votes = df.groupby('Leading Party')['Margin'].sum().sort_values(ascending=False)
leading_party_seats = df['Leading Party'].value_counts()
```

```
fig = px.bar(
    x=leading_party_seats.index[:10],
    y=leading_party_seats.values[:10],
    labels={'x': 'Party', 'y': 'number of seats'},
    title='leading parties as per seats',
)

fig.update_traces(textposition='outside')
fig.update_layout(height=1000, width=1000)
fig.show()
```



leading parties as per seats



9: Candidates with most votes

```

party_votes = df.groupby('Leading Candidate')['Margin'].sum().sort_values(ascending=False)

# Plot pie chart
plt.figure(figsize=(10, 8))
wedges, texts, autotexts = plt.pie(party_votes.head(10), labels=None, autopct='%1.1f%%', startangle=140, wedgeprops=dict(edgecolor='w'))
plt.title('Most Votes by Candidate', pad=20)
plt.axis('equal')

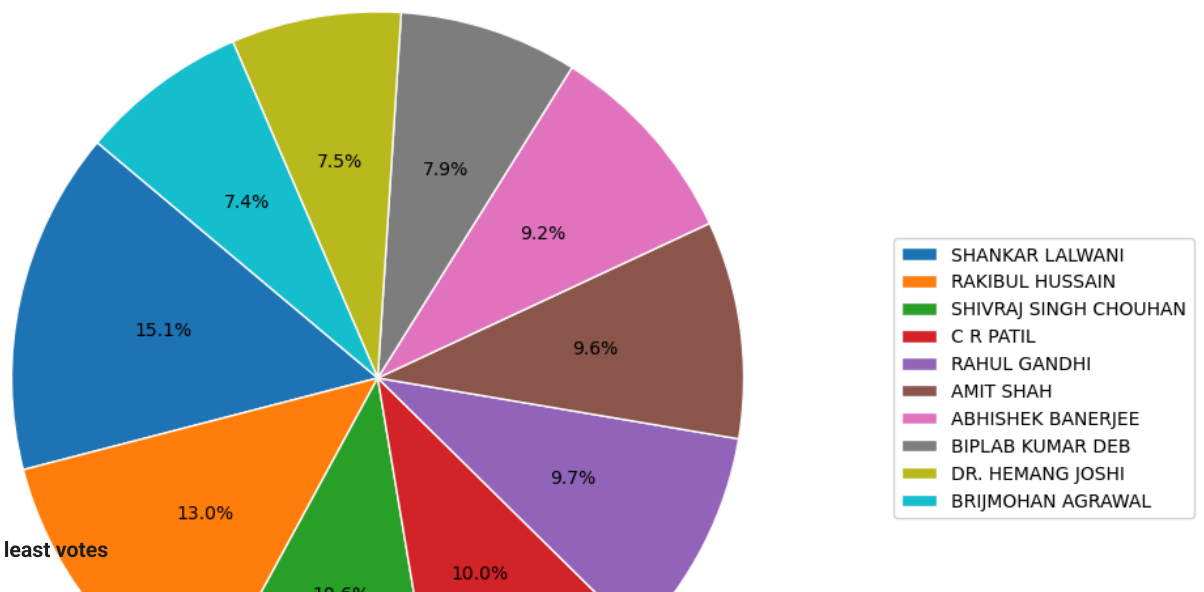
plt.legend(labels=party_votes.index, loc='center left', bbox_to_anchor=(1, 0.5), fontsize='medium')

plt.show()

```



Most Votes by Candidate



10: Candidates with least votes

```
party_votes = df.groupby('Leading Candidate')['Margin'].sum().sort_values(ascending=True)

# Plot pie chart
plt.figure(figsize=(10, 8))
wedges, texts, autotexts = plt.pie(party_votes.head(10), labels=None, autopct='%1.1f%%', startangle=140, wedgeprops=dict(edgecolor='w'))
plt.title('Least Votes by Candidate', pad=20)
plt.axis('equal')
```